DOCUMENTACION DE SISTEMA

RootCause Bitcoin Defense

17 · Resumen ejecutivo

Version analizada 0.3.0 · commit acdf4dd · generado el 2026-08-27
Documento 17 de la serie · fuente: docs/system-documentation/17-executive-summary.md

Contenido

1 · En una página
2 · La necesidad que cubre
3 · Quién lo utiliza y cómo
4 · Capacidades principales
5 · Arquitectura, en cinco líneas
6 · Estado actual
7 · Fortalezas
8 · Riesgos
9 · Oportunidades de mejora
10 · Lo que este producto demuestra sobre quien lo construyó
11 · Próximos pasos recomendados
12 · Conclusión

Qué es este sistema, qué necesidad cubre, en qué estado está y qué decisiones requiere. Escrito para quien tiene que decidir, no para quien va a programar.

---

1 · En una página

RootCause Bitcoin Defense es una aplicación de escritorio que responde a una pregunta que ninguna herramienta de custodia responde hoy: **¿de dónde vino esta clave y se puede demostrar?**

Pregunta	Respuesta
¿Qué es?	Una aplicación local que inventaría la procedencia de claves Bitcoin y detecta exposición criptográfica
¿Qué **no** es?	Una wallet. No custodia claves, no firma, no transmite y rechaza cualquier semilla que se le envíe
¿Para quién?	Titulares de custodia propia, tesorerías pequeñas, respondedores de incidentes y auditores
¿En qué estado?	Versión 0.3.0, funcional, publicada, con 64 pruebas y 11 puertas de calidad automáticas
¿Cuánto código?	Unas 6900 líneas de JavaScript, **sin una sola dependencia de terceros**
¿Licencia?	MIT

2 · La necesidad que cubre

En julio y agosto de 2026, un defecto de generación de entropía en ciertos firmwares de hardware wallets permitió reconstruir semillas creadas con esas versiones. **El protocolo Bitcoin no fue vulnerado**: los atacantes derivaron las claves y firmaron transacciones perfectamente válidas.

Eso deja un hueco que ninguna herramienta cubre:

Hecho	Consecuencia
El dispositivo puede estar desconectado y guardado, y aun así la clave está expuesta	Mirar la wallet no revela nada
Actualizar el firmware **no repara** una semilla ya generada	La reacción intuitiva no funciona
El único dato que importa es con qué firmware nació cada semilla	Y nadie lo inventaría

Este producto convierte esa lección en un control: mantiene el inventario de procedencia, lo cruza con los avisos publicados y, cuando algo coincide, explica la causa y da el runbook.

3 · Quién lo utiliza y cómo

Usuario	Qué hace con él
Titular de custodia propia	Registra sus wallets una vez y comprueba si algún aviso le afecta
Tesorería pequeña	Detecta concentración de fondos en una sola clave y quórums frágiles
Respondedor de incidentes	Obtiene hallazgos con causa raíz, evidencia y pasos de migración
Auditor	Ejecuta la auditoría de 14 etapas y obtiene un informe reproducible
Desarrollador	Lo usa como referencia de un producto de seguridad auditable de principio a fin

Se instala en Windows con un instalador que **no pide permisos de administrador**, o se descomprime y se ejecuta. No hace falta instalar nada más: el motor de ejecución viaja dentro del paquete.

4 · Capacidades principales

Capacidad	Qué significa en la práctica
26 reglas de detección en 7 familias	Procedencia de claves, arquitectura de custodia, gastos, salud del nodo, privacidad, frescura de la inteligencia y postura del despliegue
Inventario sin secretos	Solo huellas y procedencia. La aplicación rechaza activamente semillas y claves
Autoauditoría	La herramienta también se audita a sí misma: cómo está desplegada, si cifra, si vigila
La inteligencia caduca	Si la matriz de avisos lleva más de 30 días sin verificarse, eso mismo es un hallazgo

Capacidad	Qué significa en la práctica
Registro inalterable en la práctica	Cadena de hashes que hace imposible editar un evento sin que se note
Persistencia cifrada	AES-256-GCM con escritura atómica
Vigilancia opcional	Consulta periódica al nodo Bitcoin local, apagada por defecto
Auditoría en 14 etapas	Informe reproducible que declara siempre su cobertura real

---

5 · Arquitectura, en cinco líneas

Un proceso Node.js que escucha **solo en** `127.0.0.1` y sirve un panel web. Cuatro capas con dependencias hacia dentro: entrada HTTP, casos de uso, dominio —donde viven las reglas, sin saber nada de red ni de archivos— e infraestructura —cifrado, auditoría y RPC—. El estado es un único archivo cifrado en el perfil del usuario. **Cero dependencias de terceros**, verificado en cada compilación.

---

6 · Estado actual

Indicador	Valor
Versión	0.3.0, publicada con artefactos para Windows
Pruebas	64, todas pasando
Puertas de calidad automáticas	11 trabajos en 5 workflows
Sistemas operativos validados	Windows, Linux y macOS, en Node 22 y 24
Reglas implementadas y documentadas	26 de 26
Documentación	33 documentos + 26 fichas de regla + este análisis
Dependencias con vulnerabilidades	**Ninguna, porque no hay dependencias**
Deuda técnica identificada	25 hallazgos: **2 ya corregidos**; quedan 3 altos, 8 medios, 8 bajos y 4 observaciones

Verificado ejecutando la suite completa y los validadores sobre el commit `acdf4dd`.

---

7 · Fortalezas

#	Fortaleza	Por qué importa
1	**Los claims se ejecutan, no se afirman**	Un script arranca la aplicación real y le envía cinco cargas hostiles con semillas. Si alguna entrara, el build cae
2	**Cero dependencias, verificado**	Sin árbol de dependencias no hay superficie de ataque de cadena de suministro npm
3	**El producto se audita a sí mismo**	Ocho reglas miran la propia instalación. Es raro y es exactamente lo que un auditor busca

#	Fortaleza	Por qué importa
4	**La honestidad es una regla ejecutable**	Una comprobación que no se pudo hacer sale «omitida», nunca verde. Está codificado, no es una aspiración
5	**El artefacto se arranca antes de publicarse**	Se instala de verdad y se comprueba que abre **con contenido**. Atrapa el fallo clásico del binario que se instala vacío
6	**La documentación no puede divergir del código**	Una prueba compara las reglas del motor con sus fichas. Añadir una regla sin documentarla rompe el build
7	**Local por diseño, no por configuración**	Sin telemetría, sin actualización automática, sin comprobación de versión. Verificable leyendo el código
8	**Los límites están escritos**	El repositorio dedica documentos completos a lo que el producto **no** puede hacer

---

8 · Riesgos

#	Riesgo	Severidad	Naturaleza
1	El sistema no puede verificar la procedencia que se le declara	[FALLO] Alta	**Límite del dominio, no corregible.** Mitigado: omitir el dato lo convierte en hallazgo
2	Exponer el puerto invalida el modelo de seguridad	[FALLO] Alta	Decisión declarada. El propio producto lo marca como crítico
3	Un estado sin ciertas colecciones hace fallar el arranque	**Corregido**	`initialize()` repara las seis colecciones, con prueba que lo protege
4	No se comprueba la versión del esquema al cargar	[FALLO] Alta *(latente)*	Sin efecto hoy; cierto en cuanto haya una versión 2
5	Aprobaciones que declaran caducidad pero no caducan	[ALTA] Media	Puede dar falsa sensación de control
6	Parte del archivo de política no gobierna nada	[ALTA] Media	Un archivo declarativo que alguien acabará creyéndose
7	Los artefactos publicados no van firmados	[ALTA] Media	Solo checksums
8	El componente que gobierna el ciclo de vida de los incidentes no tiene prueba directa	**Corregido**	12 pruebas directas sobre `mergeIncidents`

Los 25 hallazgos, con evidencia y prioridad, en `15-risks-and-technical-debt.md`.

---

9 · Oportunidades de mejora

#	Oportunidad	Esfuerzo	Valor
1	Prueba de la concurrencia de la cola de escritura	Bajo	**Alto** — es la única defensa contra la pérdida de escrituras y nada la verifica
2	Firmar los artefactos publicados	Medio	**Alto** — coherente con un producto cuyo argumento es la procedencia verificable

#	Oportunidad	Esfuerzo	Valor
3	Hacer efectiva la caducidad de aprobaciones	Bajo	Medio
4	Aplicar o marcar como declarativos los campos de política	Bajo	Medio
5	Activar la medición de cobertura de pruebas	Muy bajo	Medio
6	Avisar cuando un valor de configuración se recorta	Muy bajo	Medio
7	Fijar la imagen base de contenedor por digest	Muy bajo	Bajo

Ninguna de estas siete requiere un cambio de arquitectura. De los cuatro hallazgos altos, uno ya está corregido, dos son límites de diseño declarados y el restante —la versión de esquema— es latente.

---

10 · Lo que este producto demuestra sobre quien lo construyó

Para un evaluador que mire el repositorio como muestra de trabajo:

Señal	Dónde se ve
Sabe distinguir un claim de una promesa	Cada afirmación pública tiene un script que la verifica en CI
Entiende el dominio, no solo la tecnología	La regla del quorum N-de-N, o la de la inteligencia que caduca, requieren saber cómo se pierde dinero de verdad
Prioriza no mentir sobre parecer completo	Documentos enteros dedicados a las limitaciones; etapas «omitidas» en vez de verdes
Piensa en quien mantendrá el código	Comentarios que explican decisiones, no líneas; generadores que impiden que dos copias divergan
Sabe dónde para su producto	«Diagnóstico primero. Intervención después.» — y no hay una sola línea de código de firma

---

11 · Próximos pasos recomendados

Plazo	Acción
Hecho	R-03 corregido y `mergeIncidents` cubierto con 12 pruebas directas
Antes del siguiente release	Prueba de concurrencia de la cola de escritura (R-10)
Próximo ciclo	Caducidad efectiva de aprobaciones, comprobación de versión de esquema, firma de artefactos
Continuo	**Verificar la matriz de avisos contra las fuentes oficiales de cada fabricante.** Es la tarea que más valor aporta y la más fácil de descuidar
Cuando toque el módulo	Cobertura de pruebas medida, avisos de configuración recortada, digest fijo de la imagen base
Decisión de producto	Si alguna vez se soporta acceso remoto, la autenticación debe preceder a esa función, no seguirla

---

12 · Conclusión

El sistema hace lo que dice hacer, y lo demuestra ejecutándolo. Su mayor virtud no es técnica sino de criterio: **prefiere declarar lo que no sabe antes que presentar un verde que no se ha ganado.** Esa disciplina está codificada en los scripts de auditoría, no confiada a la buena voluntad de quien escribe.

De los cuatro riesgos altos identificados, **uno ya está corregido** y los otros tres son, o bien límites honestos del dominio que el propio proyecto declara, o bien un defecto latente y barato de cerrar. **Ninguno cuestiona la arquitectura.**

La tarea de mantenimiento que más importa no es de código: es **mantener fresca la matriz de avisos**. Un panel en verde sobre inteligencia vieja es exactamente el problema que este producto existe para evitar — y el propio producto lo sabe, porque lo convirtió en una de sus reglas.

Siguiente	
Guía para un desarrollador nuevo	`18-new-developer-guide.md`
Matriz de trazabilidad	`19-traceability-matrix.md`
Volver al índice	`README.md`